

Software Development Fundamentals (SDF) – I

ODD 2023



Pointers and Strings

Jaypee Institute of Information Technology (JIIT), Noida

Updated by Amitesh, Dr. Ashish Mishra on 14-09-2023. If any errors or correction is required – kindly report it on Amitesh@mail.jiit.ac.in; Ashish.Mishra@mail.jiit.ac.in

Outline

- Pointers and Strings

Initializing a String

- A string can be initialized as:

```
char *s = "JIIT";
```

- Another possible way,

```
char *s;  
strcpy(s, "JIIT");
```

```
1 #include<stdio.h>
2 int main()
3 {
4     //char *s = "JIIT";
5     char s[] = "JIIT";
6     printf("%s", s);
7     return 0;
8 }
```

C:\Users\Amitesh\Desktop\JA X + v

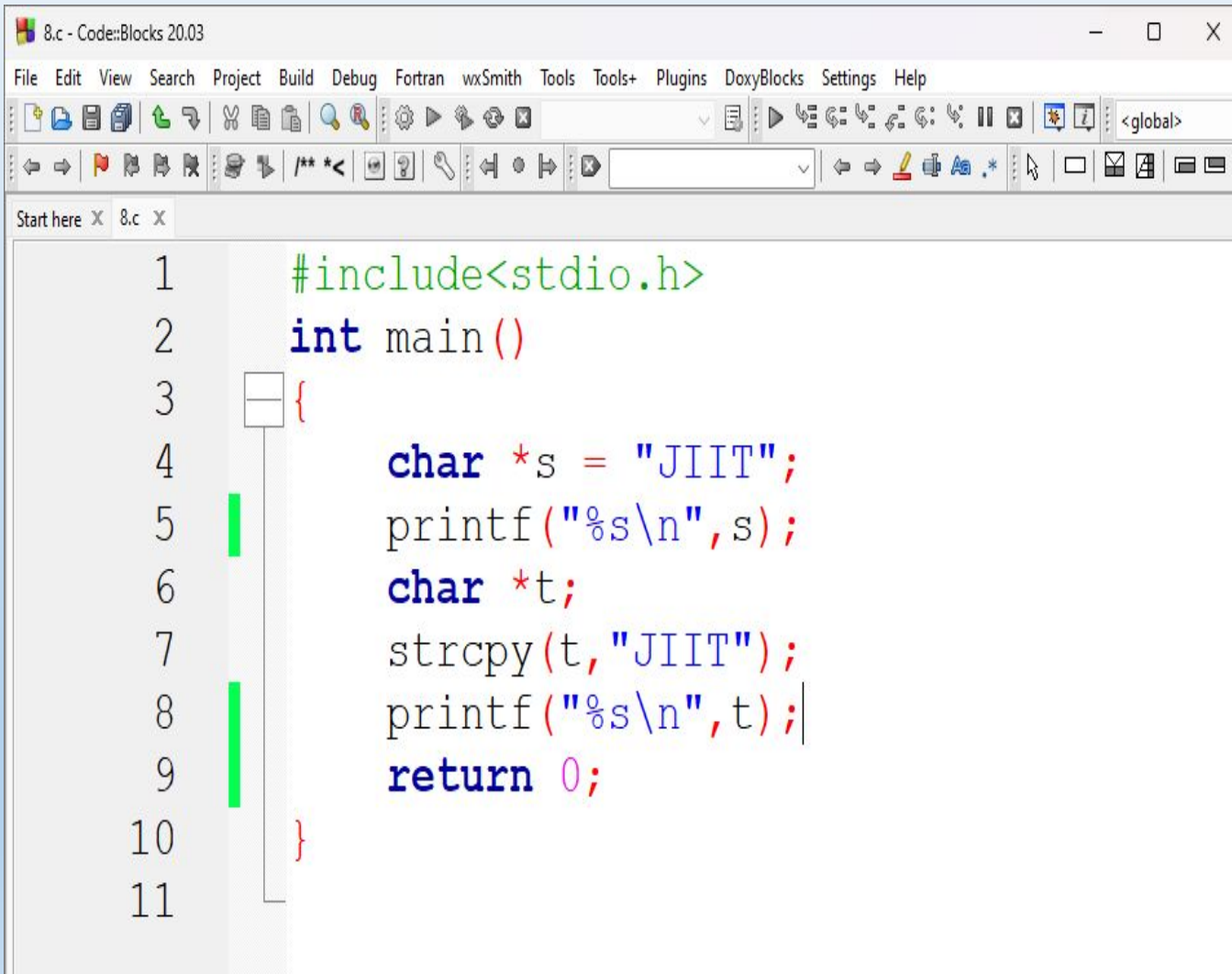
JIIT
Process returned 0 (0x0)
Press any key to continue.

```
1 #include<stdio.h>
2 int main()
3 {
4     char *s = "JIIT";
5     //char s[] = "JIIT";
6     printf("%s", s);
7     return 0;
8 }
```

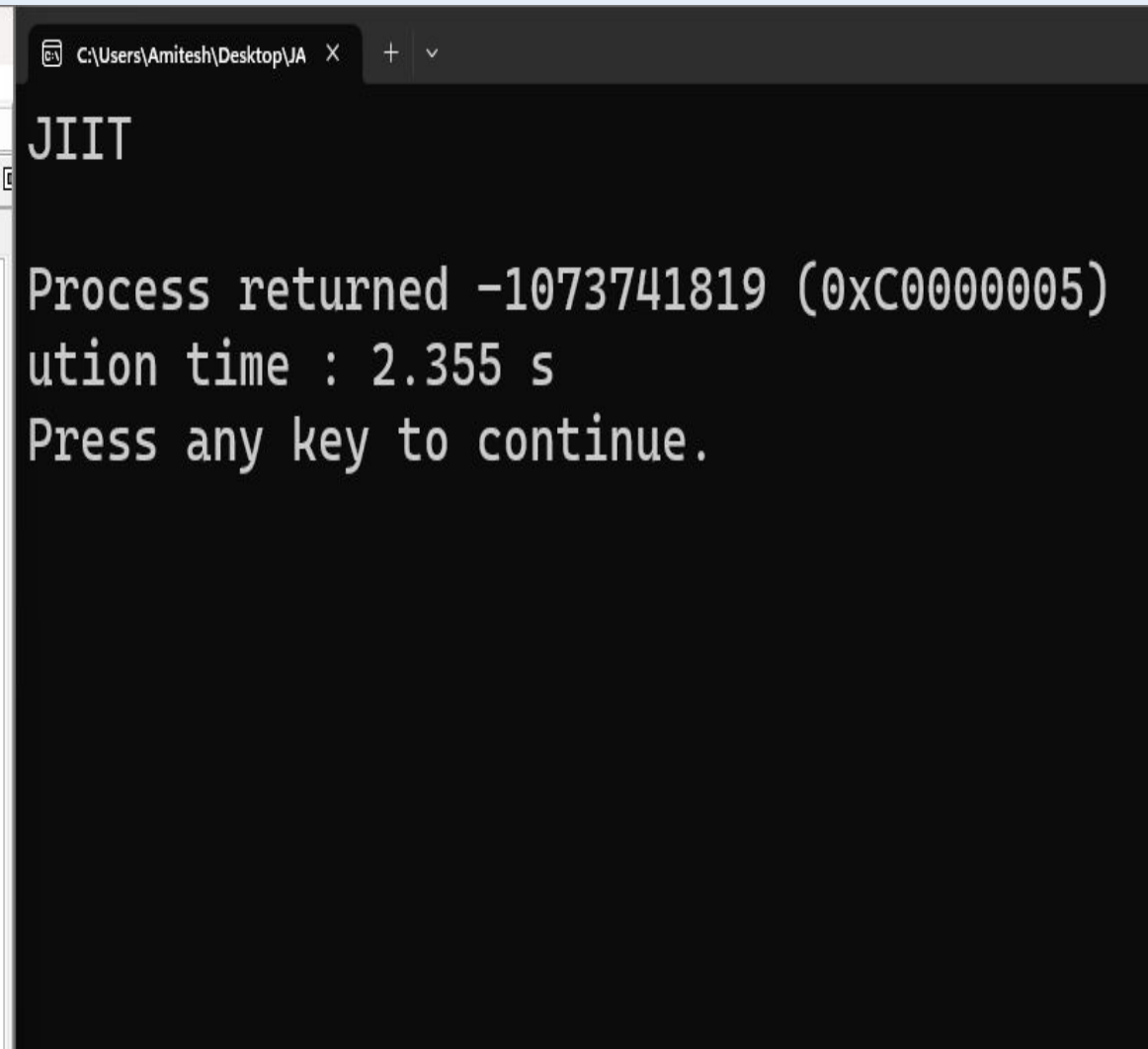
C:\Users\Amitesh\Desktop\JA X + v

JIIT
Process returned 0 (0x0)
Press any key to continue.

Activity 1: Observe the code and next slide



```
8.c - Code::Blocks 20.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
Start here x 8.c x
1  #include<stdio.h>
2  int main()
3  {
4      char *s = "JIIT";
5      printf("%s\n",s);
6      char *t;
7      strcpy(t,"JIIT");
8      printf("%s\n",t);
9      return 0;
10 }
11
```



```
C:\Users\Amitesh\Desktop\JA x + v
JIIT
Process returned -1073741819 (0xC0000005)
ution time : 2.355 s
Press any key to continue.
```

	main.c	Output
		
		
		
		
		
		
		
		
	<pre> 1 // Online C compiler to run C program online 2 #include <stdio.h> 3 4 int main() { 5 // Write C code here 6 char *s = "JIIT"; 7 printf("%s\n",s); 8 char *t; 9 strcpy(t,"JIIT"); 10 printf("%s\n",t); 11 12 return 0; 13 }</pre>	<pre> /tmp/7n9B0wCYSM.o JIIT JIIT</pre>

Home Work: Can you spot the difference in the output in the online c compiler above, and code blocks in the previous slide? Explore the reason for the difference in behaviour!!

String Using Pointer Example 1

```
#include<stdio.h>
#include<string.h>
int main()
{
    char *s;
    printf("Enter the string s\n");
    scanf("%s", s);
    printf("The input is %s\n", s);
    return 0;
}
```

```
Enter the string s
Hello World
The input is Hello
```

```
Process returned 0 (0x0)
Press any key to continue.
```

String using Pointer Example 2

```
#include<stdio.h>
#include<string.h>
int main()
{
    char *s;
    printf("Enter the string s\n");
    scanf("%[^\\n]s", s);
    printf("The input is %s\\n", s);
    return 0;
}
```

```
Enter the string s
Hello World
The input is Hello World

Process returned 0 (0x0)
Press any key to continue.
```

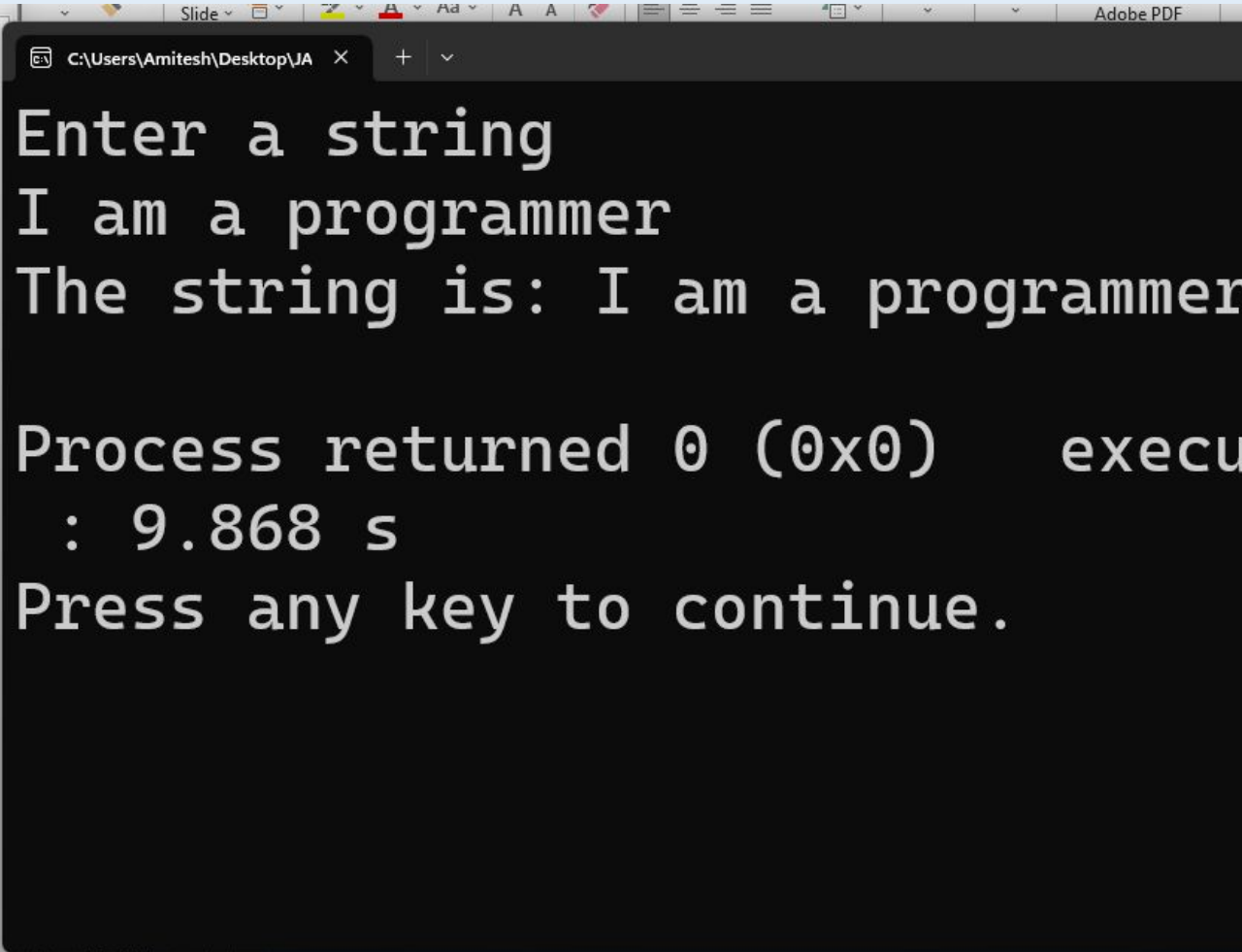
Activity 2

- Can you spot the difference between Example 1 and Example 2?
- If yes, then explore the reason for this behavioural difference

Another Way – gets and puts

```
#include<stdio.h>
#include<string.h>

int main()
{
    char arr[20];
    printf("Enter a string\n");
    gets(arr);
    printf("The string is: ");
    puts(arr);
    /*char *str;
    printf("Enter another string\n");
    gets(str);
    printf("The string is: ");
    puts(str);
    */
    return 0;
}
```



The screenshot shows a terminal window with a dark background and white text. The text displayed is the output of a C program. It starts with a prompt 'Enter a string', followed by the user input 'I am a programmer'. Then it shows 'The string is: I am a programmer'. Below this, it reports 'Process returned 0 (0x0) execution time : 9.868 s'. Finally, it prompts 'Press any key to continue.'.

```
Enter a string
I am a programmer
The string is: I am a programmer

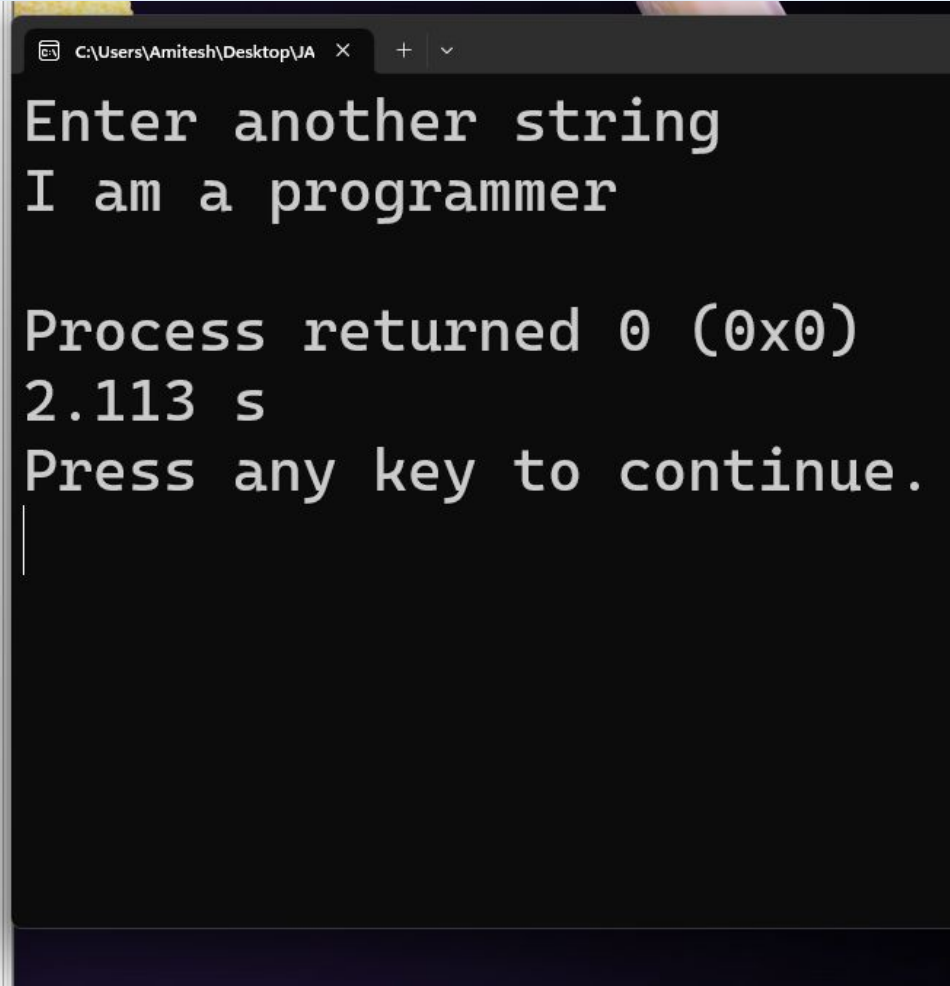
Process returned 0 (0x0)    execution time : 9.868 s
Press any key to continue.
```

Activity 3 – Observe the code and figure out what went wrong?

```
#include<stdio.h>
#include<string.h>

int main()
{
    /*char arr[20];
    printf("Enter a string\n");
    gets(arr);
    printf("The string is: ");
    puts(arr);
    */
    char *str;
    printf("Enter another string\n");
    gets(str);
    printf("The string is: ");
    puts(str);

    return 0;
}
```



```
C:\Users\Amitesh\Desktop\JA >
Enter another string
I am a programmer
```

```
Process returned 0 (0x0)
2.113 s
Press any key to continue.
```

Activity 3 – Same program on online c compiler causing seg fault!! Why ?

main.c		Run	Output	Clear
1	<code>#include<stdio.h></code>		<code>/tmp/O2wS4wfmU8.o</code>	
2	<code>#include<string.h></code>		Enter another string	
3			I am a programmer	
4	<code>int main()</code>		Segmentation fault	
5	<code>{</code>			
6	<code>/*char arr[20];</code>			
7	<code>printf("Enter a string\n");</code>			
8	<code>gets(arr);</code>			
9	<code>printf("The string is: ");</code>			
10	<code>puts(arr);</code>			
11	<code>*/</code>			
12	<code>char *str;</code>			
13	<code>printf("Enter another string\n");</code>			
14	<code>gets(str);</code>			
15	<code>printf("The string is: ");</code>			
16	<code>puts(str);</code>			
17				
18	<code>return 0;</code>			
19	<code>}</code>			
20				

char * vs const char *

1. **char *** (mutable pointer to mutable data):

char * is a pointer to a mutable (modifiable) character or character array. It means you can use this pointer to modify the data it points to.

Example

```
char str[] = "Hello";
```

```
char *ptr = str;
```

```
ptr[0] = 'J'; // This is allowed; it changes 'H' to 'J'
```

2. **const char *** (mutable pointer to constant data)

`const char *` is a pointer to constant character(s) or character array. It means you can use this pointer to access the data but not modify it.

Example:

```
const char str[] = "Hello";
```

```
const char *ptr = str;
```

```
char ch = ptr[0]; // This is allowed; it reads 'H'
```

```
ptr[0] = 'J';    // This will result in a compilation error
```

3. **char * const (constant pointer to mutable data):**

- char * const is a constant pointer to a mutable character or character array.
- It means you cannot change the pointer to point to a different memory location, but you can modify the data it points to.

- Example:

```
char str[] = "Hello";  
char *const ptr = str; // This pointer is constant  
ptr[0] = 'J';          // This is allowed; it changes 'H' to 'J'  
// ptr = some_other_pointer; // This assignment is not allowed
```

Example of methods in string.h header file

`char *strcpy(char *dest, const char *src);` copies a const string src to dest

`char *strcat(char *dest, const char *src);` concatenate a const string src to dest

`int strcmp(const char *s1, const char *s2);` compares two strings

`char *strchr(const char *s, int c);` searches a character c

`char *strstr(const char *src, const char *sub);` search the first occurrence of a substring sub in another string src

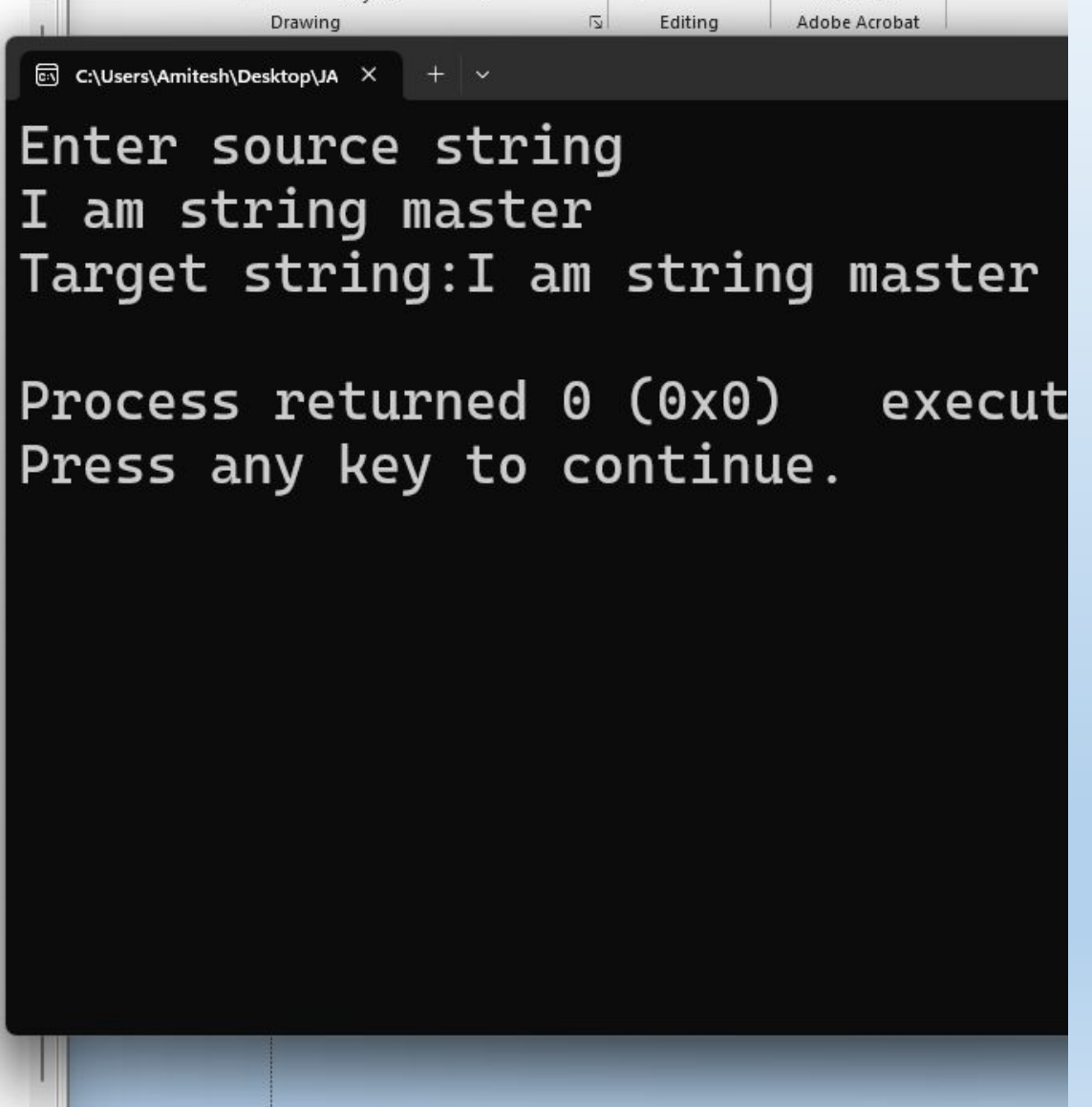
String functions usage

main.c	Output
<pre>1 #include<stdio.h> 2 #include<string.h> 3 int main() 4 { 5 const char *src = "I am a humble programmer"; 6 char *dest; 7 printf("Before copy : src=%s, dest=%s\n",src,dest); 8 strcpy(dest,src); 9 printf("After copy : src=%s, dest=%s\n",src,dest); 10 printf("strcmp usage: "); 11 if(strcmp(dest,src)==0) 12 printf("both strings are equal\n"); 13 else 14 printf("both strings are not equal\n"); 15 strcat(dest,src); 16 printf("Concatenated string = %s\n", dest); 17 char *search = strchr(src,(int)'a'); 18 printf("strchr usage: String after a is %s\n", search); 19 const char * sub="ogr"; 20 printf("Substring is %s", strstr(src, sub)); 21 return 0; 22 } 23</pre>	<pre>/tmp/tZMWXOpWav.o Before copy : src=I am a humble programmer, dest= After copy : src=I am a humble programmer, dest=I am a humble programmer strcmp usage: both strings are equal Concatenated string = I am a humble programmerI am a humble programmer strchr usage: String after a is am a humble programmer Substring is ogrammer</pre>

Copy String using Pointers

```
#include<stdio.h>
#include<string.h>
int main( )
{
    char str1[20], str2[20];
    printf("Enter source string\n");
    gets(str2);
    char *target=str1, *source=str2;
    while (*source)
    {
        *target = *source;
        source++;
        target++;
    }

    *target = '\0';
    printf("Target string:");
    puts(str1);
    return 0;
}
```



```
Enter source string
I am string master
Target string:I am string master

Process returned 0 (0x0)   execut
Press any key to continue.
```

Question

```
void main()
{
    char *x = "Hello World.";
    for( ; *x != '\0'; x++)
    {
        printf("%c", *x);
    }
    printf("\n");
}
```

References:

- <https://www.pdfdrive.com/c-in-depth-books.html>
- <https://www.programiz.com/c-programming/c-pointers>
- <https://www.w3schools.in/c-tutorial/pointers/>

End of Lecture